Jour 4 : Analyse de sentiments

Classification et interpretation des emotions dans le texte

- Travailler avec des datasets publics (IMDB)
- Implementer des classifieurs de sentiments
- Evaluer et interpreter les resultats
- Deployer un systeme de production

© 2026 UTOPIOS - Tous droits reserves

Analyse de sentiments : definition

Sentiment Analysis est la tache de determiner l'opinion ou l'emotion exprimee dans un texte. C'est une application tres importante du NLP dans le monde reel.

Applications reelles

- Analyse de retours clients
- Monitoring de marque sur les reseaux
- Analyse de critiques produits
- Monitoring de reputation
- Analyse de feedback employes
- Analyse des tweets et commentaires

Types de classification

- **Binaire :** Positif / Negatif
- **Ternaire :** Positif / Neutre / Negatif
- **5-etoiles :** 1 a 5 etoiles
- **Multi-label :** Plusieurs emotions
- **Aspect-based :** Sentiment par aspect

> **Defi majeur :** Le sarcasme, l'ironie et le contexte rendent le sentiment analysis difficile.

> "Excellent, c'est un desastre!" exprime du sarcasme negatif, pas positif.

🎞 Dataset IMDB : benchmark standard

Le **IMDB Movie Reviews Dataset** est le dataset de reference pour l'analyse de sentiments. 50,000 critiques de films avec etiquettes binaires (positif/negatif).

Caracteristiques

- 50,000 critiques totales
- 25,000 pour training
- 25,000 pour testing
- Label binaire : positif (1) / negatif (0)
- Critiques de 1-10 lignes
- Equilibre : 50/50 pos/neg

Exemples

Positif :

"Ce film est absolument magnifique. Les acteurs jouent merveilleusement bien!"

Negatif :

"Horrible ! C'est un gaspillage de temps et d'argent."

PYTHON

```python
# Charger et preparer des donnees de sentiments
import pandas as pd
from sklearn.model_selection import train_test_split

# Exemple : dataset de critiques de films
reviews = [
    "Ce film est fantastique et tres divertissant!",
    "Horrible, une perte de temps complete",
    "Excellent! A regarder absolument",
    "Nul et ennuyeux au maximum",
    "Magnifique, les acteurs jouent brillamment"
]
labels = [1, 0, 1, 0, 1]  # 1 = positif, 0 = negatif

# Diviser train/test
X_train, X_test, y_train, y_test = train_test_split(
    reviews, labels, test_size=0.2, random_state=42
)

print(f"Train: {len(X_train)}, Test: {len(X_test)}")
for review, label in zip(X_train[:2], y_train[:2]):
    print(f"Review: {review}")
    print(f"Label: {label} ({'positif' if label == 1 else 'negatif'})")
```

🧹 Preprocessing specialise pour sentiment

Le preprocessing pour sentiment analysis est similaire au Module 1, mais avec quelques ajustements importants.

> ⚠️ **Important :** En sentiment analysis, ne PAS supprimer les negations comme "ne", "pas", "ne pas". Elles sont cruciales pour exprimer l'opinion.

PYTHON

```python
# Preprocessing adapte au sentiment analysis
import re
from nltk.corpus import stopwords

def preprocess_sentiment(texte):
    # Minuscules
    texte = texte.lower()

    # Supprimer URLs, emails
    texte = re.sub(r'http\S+|www\S+|[a-z0-9]+@[a-z0-9]+\.[a-z]+', '', texte)

    # Garder les negations (!) mais supprimer autre ponctuation
    texte = re.sub(r'[^a-z\s!?]', '', texte)

    # Espaces extras
    texte = re.sub(r'\s+', ' ', texte).strip()

    # Stopwords SANS negations (important!)
    stop_words = set(stopwords.words('french'))
    stop_words.discard('ne')
    stop_words.discard('pas')
    stop_words.discard('non')

    tokens = [w for w in texte.split() if w not in stop_words]

    return ' '.join(tokens)
```

```
# Test
exemple = "Ce film n'est pas bon du tout! Horrible."
print(preprocess_sentiment(exemple))
```

> **Astuce :** Pour les emojis, vous pouvez les convertir en texte (":)" -> "smile") car ils transportent une information sentimentale importante.

Approche lexicon-based : Simple et interpretable

L'approche **lexicon-based** utilise un dictionnaire de mots positifs/negatifs pre-defini et calcule un score de sentiment.

Avantages

- Tres simple a implementer
- Tres rapide (pas d'entrainement)
- Interpretable : on sait pourquoi
- Bon pour baseline
- Fonctionne hors-domaine

Limitations

- Peu flexible
- Pas de contexte semantique
- Sarcasme non-gere
- Performance limitee
- Langue-specifique

PYTHON

```python
# Sentiment analysis avec TextBlob (lexicon-based)
```

```python
from textblob import TextBlob

texte = "Ce film est genial! J'ai adore."
blob = TextBlob(texte)

# Polarity: -1 (negatif) a +1 (positif)
print(f"Polarity: {blob.sentiment.polarity}")
print(f"Subjectivity: {blob.sentiment.subjectivity}")

# Classification simple
if blob.sentiment.polarity > 0.1:
    sentiment = "POSITIF"
elif blob.sentiment.polarity < -0.1:
    sentiment = "NEGATIF"
else:
    sentiment = "NEUTRE"

print(sentiment)
```

Cas d'usage : TextBlob est ideal pour des tests rapides et des analyses exploratoires. Pour des systemes en production, preferez des classifieurs ML/DL.

Machine Learning : Logistic Regression + TF-IDF

Combiner TF-IDF du Module 2 avec Logistic Regression donne un classifieur simple mais efficace pour le sentiment.

PYTHON

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

# Exemple de donnees (imaginaires pour ce slide)
reviews = ["excellent film!", "horrible et ennuyeux", ...]
labels = [1, 0, ...]  # 1=positif, 0=negatif

# 1. Vectorisation TF-IDF
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
X = vectorizer.fit_transform(reviews)

# 2. Split train/test
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2)

# 3. Entrainer Logistic Regression
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)

# 4. Prediction
y_pred = clf.predict(X_test)
y_pred_proba = clf.predict_proba(X_test)  # Probabilites

# 5. Evaluation
accuracy = accuracy_score(y_test, y_pred)
precision, recall, f1, _ = precision_recall_fscore_support(y_test, y_pred,
average='weighted')

print(f"Accuracy: {accuracy:.2%}")
```

```
print(f"Precision: {precision:.2%}, Recall: {recall:.2%}")
```

Resultat typique : Avec TF-IDF + Logistic Regression sur IMDB, on obtient ~87-89% d'accuracy. C'est une tres bonne baseline.

Embeddings et representations denses

Au lieu de TF-IDF sparse, utiliser des embeddings denses (Word2Vec, FastText) capture mieux la semantique et le contexte.

Approches d'embeddings

- **Word2Vec :** Skip-gram, CBOW
- **FastText :** Subword info
- **GloVe :** Global co-occurrence
- **Pre-trained :** Wikipedia, Crawl
- **Custom :** Entrainer sur domaine

Avantages vs TF-IDF

- Capture la semantique
- Mots similaires ont embeddings proches
- Dimension reduite (100-300 vs 5000+)
- Travaille sur mots hors vocab
- Meilleure performance (~88-92%)

PYTHON

```python
# Word2Vec et FastText pour embeddings
from gensim.models import Word2Vec, FastText
from sklearn.linear_model import LogisticRegression
import numpy as np
```

```python
# Donnees tokenizees
sentences = [
    ["ce", "film", "est", "fantastique"],
    ["horrible", "perte", "temps"],
    ["excellent", "a", "regarder"]
]

# 1. Entrainer Word2Vec
w2v_model = Word2Vec(
    sentences=sentences,
    vector_size=100,  # Dimension
    window=5,  # Context window
    min_count=1,  # Min frequency
    workers=4
)

# 2. Convertir documents en vecteurs moyens
def get_document_vector(tokens, model):
    vectors = [model.wv[token] for token in tokens if token in model.wv]
    return np.mean(vectors, axis=0) if vectors else np.zeros(100)

X_train = np.array([get_document_vector(s, w2v_model) for s in sentences])

# 3. Entrainer classifier sur embeddings
clf = LogisticRegression()
clf.fit(X_train, [1, 0, 1])  # Labels

# 4. Prediction
test_tokens = ["magnifique", "film"]
test_vector = get_document_vector(test_tokens, w2v_model)
prediction = clf.predict([test_vector])[0]
print(f"Prediction: {prediction}")  # 1 = positif
```

> **FastText vs Word2Vec :** FastText gere mieux les mots hors-vocab (OOV) en utilisant des subwords. Recommande pour texte bruite.

🔄 Transfer Learning : Modeles pre-entraines

Au lieu d'entrainer un modele de zero, on peut utiliser des embeddings ou des modeles pre-entraines (BERT, GPT) et fine-tuner.

🚀 Approches modernes

Modele	Approche	Performance	Complexite
BERT	Transformer pre-trained	~95%	Haute
RoBERTa	BERT ameliore	~96%	Haute
DistilBERT	BERT compresse	~94%	Moyenne
FastText + MLP	Embeddings + NN	~90%	Basse

PYTHON

```python
# Fine-tuning BERT pour sentiment avec HuggingFace
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import TextClassificationPipeline

# Charger un modele BERT pre-trained francais
model_name = "distilbert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name,
num_labels=2)

# Pipeline for quick inference
classifier = TextClassificationPipeline(model=model, tokenizer=tokenizer)

# Prediction
result = classifier("Ce film est fantastique!")
print(result)  # [{'label': 'POSITIVE', 'score': 0.998}]
```

> **Recommandation :** Pour production, utilisez HuggingFace Transformers. C'est l'etat de l'art et relativement simple a mettre en place.

Evaluation et interpretabilite

L'evaluation d'un classifieur de sentiment va au-dela de l'accuracy. Il faut considerer precision, recall, F1-score, et la confusion.

Metriques importantes

- **Accuracy :** % correct global
- **Precision :** Parmi les positifs predits, combien sont justes ?
- **Recall :** Parmi les positifs reels, combien detected ?
- **F1-score :** Harmonic mean de precision et recall
- **AUC-ROC :** Courbe d'evaluation

Interpretabilite

- Quels mots influencent la prediction?
- Attention weights (pour Transformers)
- LIME : explanations locales
- Erreur analysis : quand se trompe-t-on?
- Test d'adversaires : robustesse

PYTHON

```python
# Matrice de confusion et rapport
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
import matplotlib.pyplot as plt
```

```
# Predictions
y_pred = model.predict(X_test)
y_pred_proba = model.predict_proba(X_test)[:, 1]

# Rapport detaille
print(classification_report(y_test, y_pred, target_names=['Negatif', 'Positif']))

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
print(cm)

# AUC-ROC
auc = roc_auc_score(y_test, y_pred_proba)
print(f"AUC-ROC: {auc:.4f}")
```

Cas d'usage specifiques :

- Si peu de false positifs importants → priorite a precision
- Si on veut detecter tous les sentiments negatifs → priorite a recall

⚠ Challenges et cas difficiles

Certains types de texte sont particulierement difficiles pour le sentiment analysis.

Defis courants

- **Sarcasme :** "Genial, ca marche sur 1000 appareils"
- **Negation :** "N'est PAS bon" vs "Est bon"
- **Contexte :** Memes mots, sentiments differents
- **Aspect-based :** "Bon film, mauvais sous-titres"
- **Mixed sentiment :** "Aime le scenario, hate les acteurs"

Solutions

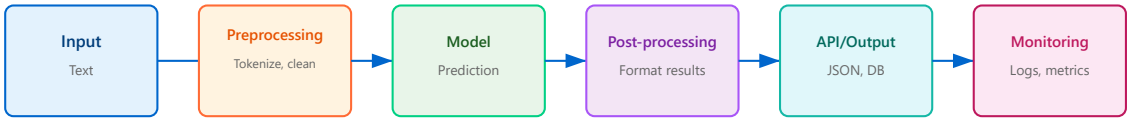
- Utiliser des modeles contextuels (BERT)
- Augmenter les donnees d'entrainement
- Fine-tuner sur votre domaine specifique
- Detecter et gerer le sarcasme separement
- Analyser par aspect quand possible

⚠ Limitation humaine : Meme les humans n'accordent pas toujours sur le sentiment d'un texte. L'inter-annotator agreement est souvent 85-90%. On ne peut pas faire mieux que ca!

Deploiement en production

Passer d'un notebook a un systeme de production necessite plusieurs etapes.

Pipeline de production typique



PYTHON

```python
# Production API avec Flask + FastText
from flask import Flask, jsonify, request
from gensim.models import FastText
from sklearn.linear_model import LogisticRegression

app = Flask(__name__)

# Charger les modeles
fasttext_model = FastText.load('fasttext_model.bin')
classifier = load('classifier.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    text = data.get('text')

    # Preprocessing (same as training)
    text_clean = preprocess(text)

    # Embedding
    embedding = fasttext_model.infer_vector(text_clean.split())

    # Prediction
```

```
    sentiment = classifier.predict([embedding])[0]
    confidence = classifier.predict_proba([embedding])[0].max()

    return jsonify({
        'text': text,
        'sentiment': 'POSITIVE' if sentiment == 1 else 'NEGATIVE',
        'confidence': float(confidence)
    })

if __name__ == '__main__':
    app.run(debug=False, port=5000)
```

Conclusion et perspectives

Ce que nous avons couvert

- Definition et applications du sentiment analysis
- Dataset IMDB benchmark
- Approaches lexicon-based (TextBlob)
- ML classique (TF-IDF + LogReg)
- Deep Learning (LSTM, Transformers)
- Evaluation et interpretabilite
- Deployment en production

Prochaines etapes

- Aspect-based sentiment analysis
- Multilingue (Multilingual BERT)
- Emotion detection (>2 classes)
- Sarcasm detection specialise
- Continual learning et adaptation
- Integration avec knowledge graphs

Summary : Le sentiment analysis est un domaine mature avec de nombreuses techniques

disponibles. Pour 90% des cas, une approche TF-IDF + LogisticRegression ou BERT fine-tune suffira. L'important est d'adapter a votre contexte specifique et de tester systematiquement.

Ressources :

- HuggingFace Models Hub : Pre-trained models
- IMDB Dataset : http://ai.stanford.edu/~amaas/data/sentiment/
- Kaggle : Hundreds of sentiment datasets
- Papers : Read about latest approaches (BERT, GPT, etc.)